```
mport required libraries

ort numpy as np
ort matplotlib.pyplot as plt
ort tensorflow as tf

m tensorflow.keras.datasets import mnist
m tensorflow.keras.models import Sequential
m tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
m tensorflow.keras.utils import to_categorical

m sklearn.metrics import classification_report, confusion_matrix
ort seaborn as sns

nt("All libraries imported successfully!")
nt("TensorFlow version:", tf.__version__)
```

```
All libraries imported successfully!
TensorFlow version: 2.21.0
```

```
# Load the MNIST dataset

(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Dataset loaded successfully!")
print("Training images shape:", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing images shape:", X_test.shape)
print("Testing labels shape:", y_test.shape)
```

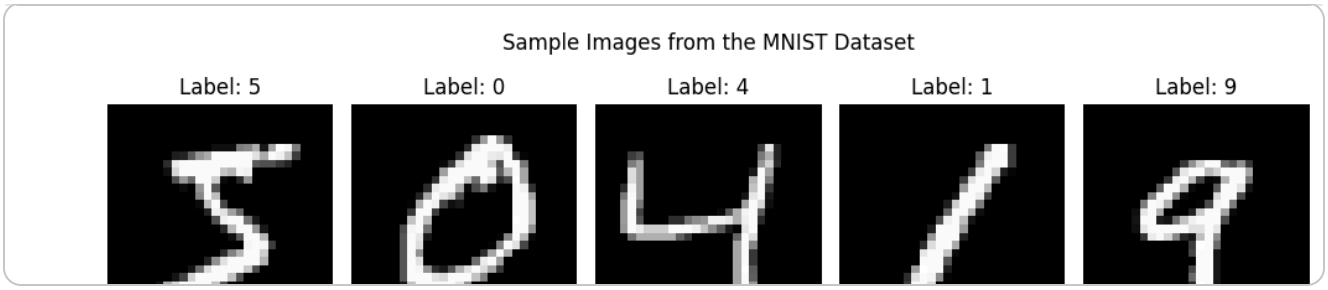
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Dataset loaded successfully!
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)
```

```
# Display sample handwritten digits

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.suptitle("Sample Images from the MNIST Dataset")
plt.tight_layout()
plt.show()
```




```
# Preprocess the images

# Normalize pixel values between 0 and 1
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Add a channel dimension for the CNN
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Convert labels into categorical format
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

print("Image preprocessing completed!")
print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)
print("Training labels:", y_train_cat.shape)
print("Testing labels:", y_test_cat.shape)
```

```
Image preprocessing completed!
Training images: (60000, 28, 28, 1)
Testing images: (10000, 28, 28, 1)
Training labels: (60000, 10)
Testing labels: (10000, 10)
```

```
# Build the CNN model

model = Sequential([
    Conv2D(32, (3, 3), activation="relu",
           input_shape=(28, 28, 1)),

    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation="relu"),

    MaxPooling2D((2, 2)),

    Flatten(),

    Dense(128, activation="relu"),

    Dropout(0.3),

    Dense(10, activation="softmax")
])

# Compile the model
model.compile(
```

```
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Display model architecture
model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/ba
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param
conv2d (Conv2D)	(None, 26, 26, 32)	3
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,4
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	204,9
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 10)	1,2

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

```
# Train the CNN model

history = model.fit(
    X_train,
    y_train_cat,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

print("Model training completed successfully!")
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 45s 103ms/step - accuracy: 0.9218 - loss
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 101ms/step - accuracy: 0.9769 - loss
```

```
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 44s 104ms/step - accuracy: 0.9831 - loss
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 101ms/step - accuracy: 0.9870 - loss
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 101ms/step - accuracy: 0.9892 - loss
Model training completed successfully!
```

```
# Evaluate the model on test images

test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test_cat,
    verbose=1
)

print("\nMODEL EVALUATION RESULTS")
print("------------------------")
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9920 - loss: 0

MODEL EVALUATION RESULTS
------------------------
Test Loss: 0.0246
Test Accuracy: 99.20%
```

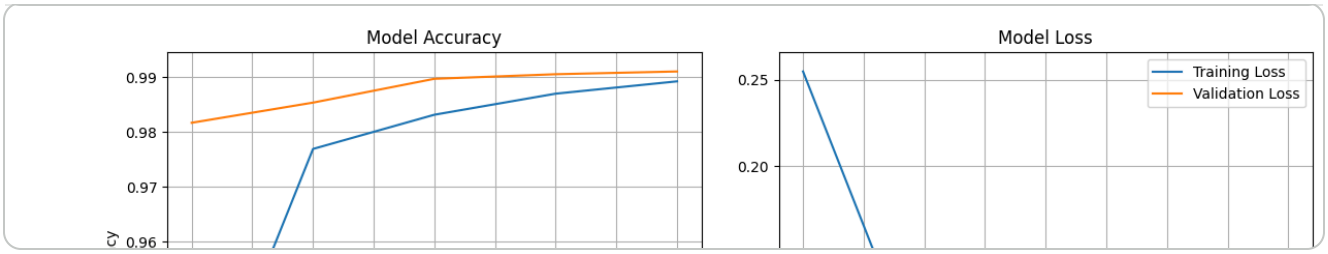
```
# Plot training and validation accuracy and loss

plt.figure(figsize=(12, 5))

# Accuracy graph
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Loss graph
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# Generate predictions for test images

y_prob = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)
y_true = y_test

# Display classification report
print("CLASSIFICATION REPORT")
print(classification_report(y_true, y_pred, digits=4))

# Create confusion matrix
cm = confusion_matrix(y_true, y_pred)

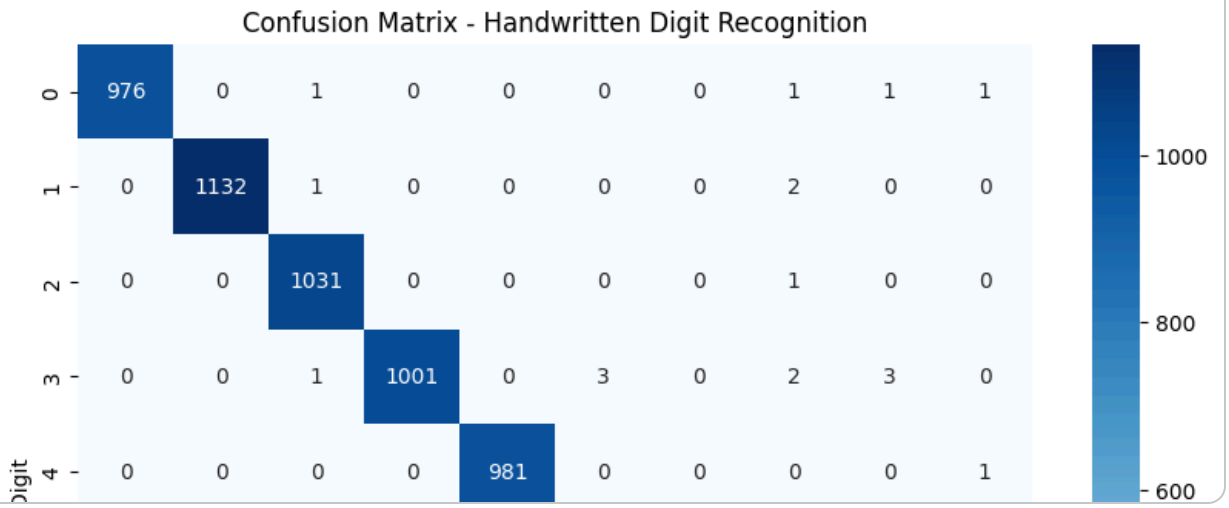
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=range(10),
    yticklabels=range(10)
)

plt.title("Confusion Matrix - Handwritten Digit Recognition")
plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")
plt.show()
```

```
CLASSIFICATION REPORT
              precision    recall  f1-score   support

           0     0.9929    0.9959    0.9944       980
           1     0.9947    0.9974    0.9960      1135
           2     0.9894    0.9990    0.9942      1032
           3     0.9921    0.9911    0.9916      1010
           4     0.9909    0.9990    0.9949       982
           5     0.9854    0.9865    0.9860       892
           6     0.9979    0.9833    0.9905       958
           7     0.9893    0.9932    0.9913      1028
           8     0.9888    0.9938    0.9913       974
           9     0.9980    0.9792    0.9885      1009

    accuracy                         0.9920     10000
   macro avg     0.9919    0.9918    0.9919     10000
weighted avg     0.9920    0.9920    0.9920     10000
```




```
# Display sample predictions

plt.figure(figsize=(12, 6))

for i in range(15):
    plt.subplot(3, 5, i + 1)

    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")

    actual = y_test[i]
    predicted = y_pred[i]

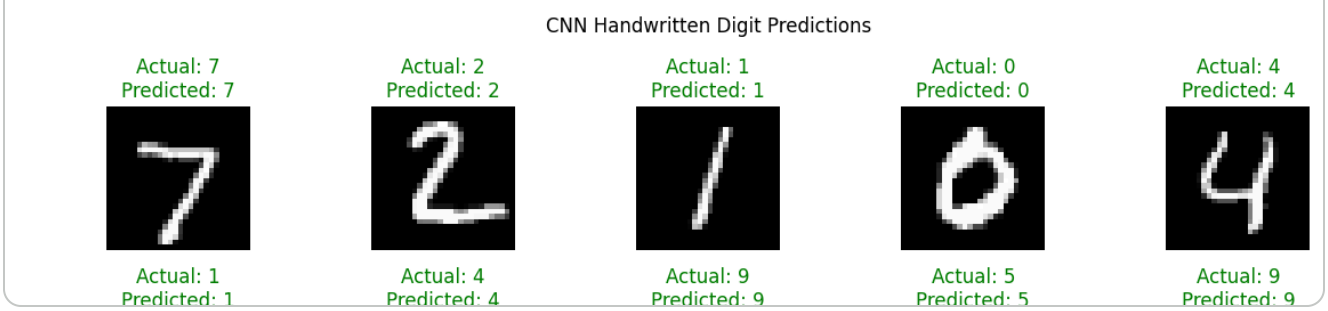
    color = "green" if actual == predicted else "red"

    plt.title(
        f"Actual: {actual}\nPredicted: {predicted}",
        color=color
    )

    plt.axis("off")

plt.suptitle("CNN Handwritten Digit Predictions")
```

```
    plt.tight_layout()
    plt.show()
```



Handwritten Digit Recognition Using Deep Learning

1. Introduction

Handwritten digit recognition is an application of deep learning that identifies handwritten numbers from images. It can be used in applications such as digit recognition in forms and automated number processing.

2. Objective

The objective of this project is to develop a Convolutional Neural Network (CNN) that recognizes handwritten digits from 0 to 9.

3. Dataset

The project uses the MNIST dataset, which contains grayscale images of handwritten digits. Each image has a resolution of 28 × 28 pixels.

4. Technologies Used

Python
TensorFlow and Keras
NumPy
Matplotlib
Seaborn
Scikit-learn
Google Colab

5. Data Preprocessing

The image pixel values were normalized from 0–255 to 0–1. The images were reshaped to include a channel dimension, and the digit labels were converted into categorical format.

6. Model Architecture

A Convolutional Neural Network was developed using:

Two convolutional layers
Two max-pooling layers
A flattening layer
A dense layer
A dropout layer
An output layer with 10 neurons

The Adam optimizer and categorical cross-entropy loss function were used for training.

7. Model Training

The model was trained for five epochs using a batch size of 128. Ten percent of the training data was used for validation.

8. Model Evaluation

The model was evaluated using test accuracy, test loss, precision, recall, and F1-score. A confusion matrix was generated to analyze classification performance. Training and validation accuracy and loss graphs were also created.

9. Results and Discussion

The CNN learned to recognize handwritten digits from the MNIST dataset. Its actual test accuracy, loss, and classification metrics are available in the notebook outputs.

10. Conclusion

This project demonstrates how deep learning and CNNs can be applied to handwritten

```
 digit recognition. The results illustrate how a trained neural network can classi
fy images into ten digit categories, from 0 to 9.
```

Handwritten Digit Recognition Using Deep Learning

1. Introduction Handwritten digit recognition is an application of deep learning that identifies handwritten numbers from images. It can be used in applications such as digit recognition in forms and automated number processing.
2. Objective The objective of this project is to develop a Convolutional Neural Network (CNN) that recognizes handwritten digits from 0 to 9.
3. Dataset The project uses the MNIST dataset, which contains grayscale images of handwritten digits. Each image has a resolution of 28 × 28 pixels.
4. Technologies Used Python TensorFlow and Keras NumPy Matplotlib Seaborn Scikit-learn Google Colab
5. Data Preprocessing The image pixel values were normalized from 0–255 to 0–1. The images were reshaped to include a channel dimension, and the digit labels were converted into categorical format.
6. Model Architecture A Convolutional Neural Network was developed using: Two convolutional layers Two max-pooling layers A flattening layer A dense layer A dropout layer An output layer with 10 neurons The Adam optimizer and categorical cross-entropy loss function were used for training.
7. Model Training The model was trained for five epochs using a batch size of 128. Ten percent of the training data was used for validation.
8. Model Evaluation The model was evaluated using test accuracy, test loss, precision, recall, and F1-score. A confusion matrix was generated to analyze classification performance. Training and validation accuracy and loss graphs were also created.
9. Results and Discussion The CNN learned to recognize handwritten digits from the MNIST dataset. Its actual test accuracy, loss, and classification metrics are